



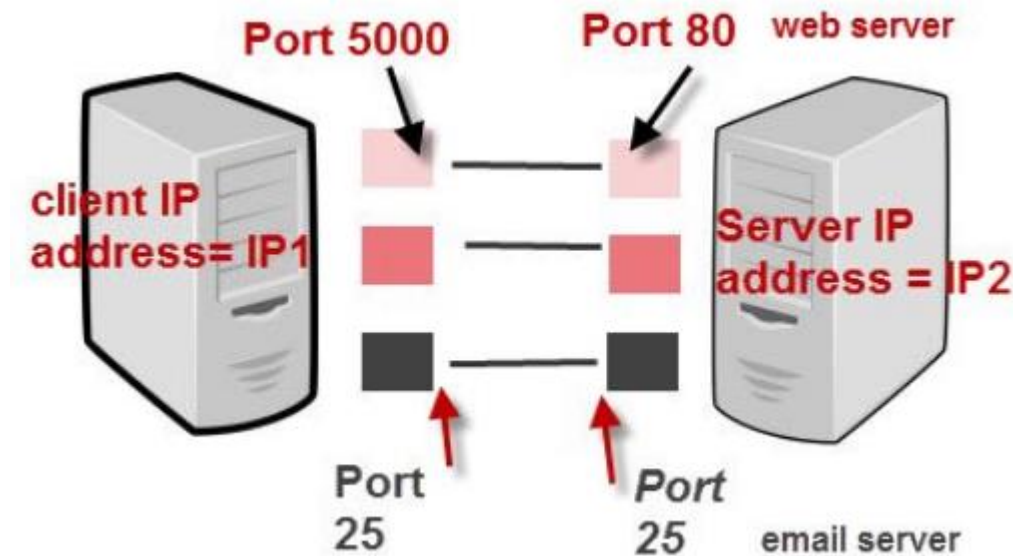
UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 2.1:TCP / UDP



Sockets

- They use the same actions as any other I/O functions: Read and write
- Two main types for this course: **TCP** and **UDP**
- Need an **Address** and a **Port**
- Sockets are endpoints in the communication between two processes in the same or different machines.



Sockets

- InterNetwork -> IPv4 family of addresses
- InterNetworkV6 -> IPv6 family of addresses
- Dgram vs. Stream -> independent packets (Datagrams) vs continuous stream of data
- Dgram always for UDP.
- Stream always for TCP.
- `newSocket.Shutdown()` -> shutdown the transfer of data, but keep socket alive
- `newSocket.Close()` -> Stop transfers and destroy socket

```
newSocket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.Udp);
```

```
newSocket = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
```

Adresses

- We need an Address to connect two processes!
- IP:
 - String
 - Used for source and destination of data
 - IPAddress.Any means any IP address is valid
- Port:
 - Int
 - Used for connecting to each application

```
IPEndPoint ipep = new IPEndPoint(IPAddress.Any,port);  
or  
IPEndPoint ipep = new IPEndPoint(IPAddress.Parse("192.168.1.45"),port);
```

Binding

- Binding is the process of assigning an IP Address to a Socket.
- Binding is only necessary to receive data from arbitrary hosts (otherwise, the socket doesn't need to know!)
- Sometimes, if a socket was not properly closed, calling Bind with the same IP/Port will return a SocketError indicating that the address is already in use.

```
newSocket.Bind(ipep);
```

UDP Sockets

- data: bytes[] Data to receive or be sent
- Remote: EndPoint, destination or source of the data
- recv: Int, size of the data
- SocketFlags.None : Special options. leave as None, for now

```
recv = newSocket.ReceiveFrom(data, ref Remote);
```

```
newSocket.SendTo(data, recv, SocketFlags.None, Remote);
```

- UDP does not check if sent packets are received
- In UDP, no guarantee that the order of packets sent is preserved at destination
- SendTo and ReceiveFrom are blocking functions

TCP Sockets (1/2)

- Listen(Backlog): Sets the socket in listening mode, and accepts up to N connections
- Accept(): returns a client socket that has tried to connect. Returns error if used in non-blocking mode with no pending connections
- Listen and Accept are done on the server side.
- Connect(ipEndPoint): tries to connect to a specific endpoint (ipep). Is blocking.

```
newSocket.Listen(10);
```

```
client = newSocket.Accept()
```

```
newSocket.Connect(ipep);
```

TCP Sockets (2/2)

- Arguments are the same as in UDP
- Receive returns error when:
 - Socket.ReceiveTimeout or Socket.SendTimeout expired
 - No connection available (handshake was interrupted or never started)
 - Data is larger than buffer
- Receive returns 0 if client is disconnected
- Both return #bytes sent/received
- It's the dev's responsibility to ensure that the all commands are sent safely (check connection, check what the message means, check what the error means...)

```
client.Send(data, recv, SocketFlags.None);
```

```
recv = client.Receive(data);
```

TCP vs. UDP

- TCP ensures connection and messages are consistent at all times
- TCP reads the messages in order of arrival (blocking)
- UDP eliminates previous messages from the buffer when a new one arrives
- UDP doesn't use connections neither handshakes (less safe)
- UDP typically used when you need the most updated data available
- TCP typically used when you need reliable communication between host and client



TO DO — TCP Edition

TO DO 1

- Create and bind the socket
 - Any IP that wants to connect to the port 9050 using TCP will communicate with this socket
 - Don't forget to set the socket in listening mode

```
void Socket(AddressFamily addressFamily, SocketType socketType, ProtocolType protocolType);  
public void Bind(EndPoint localEp);  
socket.Listen(10);
```

TO DO 2

- Connect the client
 - Create the server endpoint so we can try to connect to it. You'll need the server's IP and the port we binded it to before
- When calling connect and succeeding, our server socket will create a connection between this endpoint and the server's endpoint

```
public IPEndPoint(IPAddress address, int port);  
public void Connect(EndPoint remoteEP);
```

TO DO 3

- TCP makes managing connections easy, so we are going to put it to use.
- Accept any incoming clients and store them in a user. When accepting, we can now store a copy of our server socket who has established a communication between a local endpoint (server) and the remote endpoint(client)

```
public Socket Accept();
```

TO DO 4

- Using the socket that stores the connection between the 2 endpoints, call the TCP send function with an encoded message;

```
public int Send(byte[] buffer);
```

Conversion String-bytes / bytes-string

```
Conversion string-bytes / bytes-string  
data = Encoding.ASCII.GetBytes(text);
```

```
text = Encoding.ASCII.GetString(data,0,recv);
```

TO DO 5

- Receiving messages
 - Create an infinite loop to start receiving messages for this user
 - You'll have to use the socket method `receive()` to be able to get them.

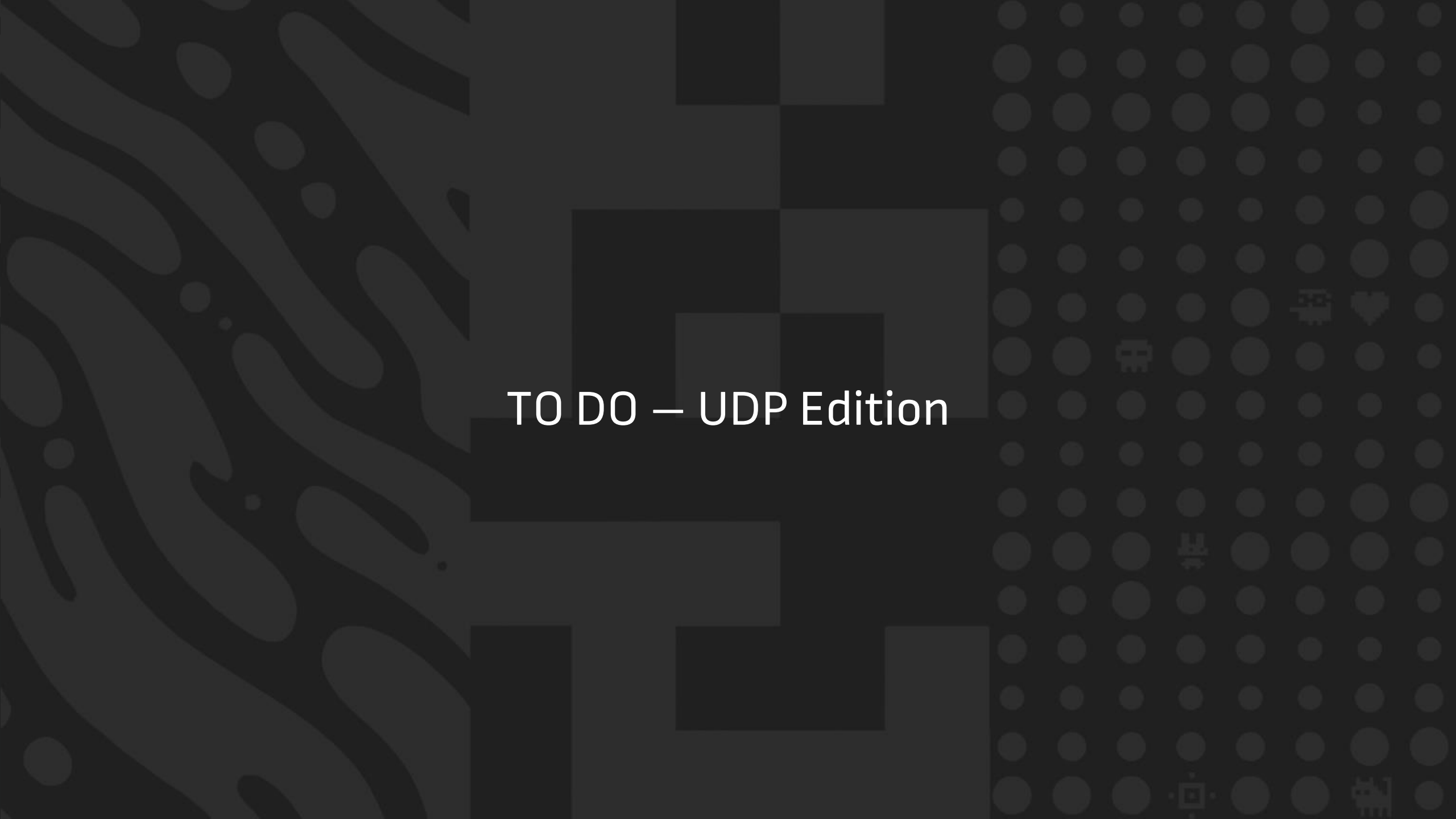
```
public int Receive(byte[] buffer);
```

TO DO 6

- Answering with the server
 - We'll send a ping back every time a message is received
 - Start another thread to send a message, same parameters as this one.
 - We'll use the socket we stored as a user to send a "ping". Just call the same send function as before and encode the string "ping"

TO DO 7

- Receiving the ping on the client
 - Similar to what we already did with the server, we have to call the `Receive()` method from the socket.

The background is a dark gray composition of three distinct patterns. On the left, there are large, flowing, organic shapes in a slightly lighter shade of gray. In the center, a grid of squares of varying shades of gray is visible. On the right, a dense pattern of small, light gray dots covers the area, with a few small, faint icons interspersed.

TO DO — UDP Edition

TO DO 1

- Create and bind the socket
 - UDP doesn't keep track of our connections like TCP. This means that we "can only" reply to other endpoints /since we don't know where or who they are.
 - We want any UDP connection that wants to communicate with 9050 port to send it to our socket. So as with TCP, we create a socket and bind it to the 9050 port.

```
public IPEndPoint(IPAddress address, int port);  
void Socket(AddressFamily addressFamily, SocketType socketType, ProtocolType protocolType);  
public void Bind(EndPoint localEp);
```

TO DO 2

- Send a message to the server
 - Unlike with TCP, we don't "connect" first, we are going to send a message to establish our communication so we need an endpoint
 - We have to use the server's IP and the port we've binded it to. initialize the endpoint with the server information so we can connect to it.

```
public IPEndPoint(IPAddress address, int port);  
void Socket(AddressFamily addressFamily, SocketType socketType, ProtocolType protocolType);
```

TO DO 2.1

- Send a message to the server
 - Create a new socket with the correct protocol to use it to send the Handshake to the server's endpoint. This time, our UDP socket doesn't have that information, so we have to pass it as a parameter on it's SendTo() method

```
public int SendTo(byte[] buffer, int size, SocketFlags socketFlags, EndPoint remoteEP);
```

TO DO 3

- Receive the message and manage the connection
 - We don't know who may be communicating with this server, so we have to create an endpoint with any address and a remote so we can reply to it later. Use `receivefrom` to block the thread until receiving the message,

```
IPEndPoint sender = new IPEndPoint(IPAddress.Any, 0);  
EndPoint Remote =(EndPoint)(sender );  
public int ReceiveFrom(byte[] buffer, ref EndPoint remoteEP);
```

TO DO 4

- Answering with the server
 - When our UDP server receives a message from a random remote, it has to send a ping.
 - Use `socket.SendTo` to send it using the remote we stored earlier.

```
public int SendTo(byte[] buffer, int size, SocketFlags socketFlags, EndPoint remoteEP);
```

TO DO 5

- Receive the ping
 - Same as in the server, in this case the remote is a bit useless, since we already know it's the server who's communicating with us

The background is a dark gray composition of three distinct patterns. On the left, there are large, flowing, organic shapes in a slightly lighter shade of gray. In the center, a grid of squares of varying sizes is visible, some appearing as darker gray blocks against the lighter background. On the right side, a dense pattern of small, light gray dots covers the area, with a few small, faint icons interspersed among them.

Other useful material

Working Locally

- We can work executing both processes in the same machine, referring to itself with the IP address 127.0.0.1 (localhost).
- You will have to launch two instances of the application (the client, and the server) to make the test.

Working in Class

- Your computers should have their IPs written on their cover.
- To know your IP, run “ipconfig” in a terminal, and choose the connection that you are using.
- From any computer you can check if you could connect to any other computer by running “ping 192.0.0.0” (where 192.0.0.0
- should be changed by your target IP).
- Beware of the firewalls: if connection about 2 computers doesn't work, but it works between two other computers, re-configure
- the Windows Firewall.

Try-Catch example

- Try: Attempt something unless an error is raised
- Catch: If the previous try block fails, don't stop the program, but do something else (i.e. fail gracefully)
- Usually, print the error (System.Exception e) and do something to make sure nothing is "broken" because of the previous failure.
- Remember: If the "Catch" block executes, nothing in the "Try" block may have executed (no variables created/modified, no functions run)

```
try
{
    newSocket.Listen(10);
    Debug.Log("Waiting for clients...");
    client = newSocket.Accept();
    clientep = (IPEndPoint)client.RemoteEndPoint;
    Debug.Log("Connected: "+clientep.ToString());
    connected=true;
}
catch (System.Exception e)
{
    Debug.Log("Connection failed.. trying again...");
}
```

Working Remotely

- Working Remotely
- If you want to connect with a remote machine (a classmate's machine), you will have to copy your application to the other machine.
- If you need to know your public IP address (your router's IP address publicly visible in the internet), just type "my ip address" in google.
- Of course you will not be interested in sending any data packets to your classmate's router, but his/her machine in the local network. You will have to configure your router to forward packets coming into a certain port number to the desired machine. (of your choice, but some high number such as 9999 for instance). Different routers have different interfaces, you will have to investigate on how to do that. Here's some
- reference: <https://www.noip.com/support/knowledgebase/general-port-forwarding-guide/>
- Spoiler Alert: We will see how to overcome the problems derived from the NAT layer in your Router, and they exist, in the future.

Examples and other documentation

- Client / Server simple examples:
- <http://www.java2s.com/Code/CSharp/Network/SimpleUdpClient.htm>
- <http://www.java2s.com/Code/CSharp/Network/SimpleUdpServer.htm>
- <http://www.java2s.com/Code/CSharp/Network/SimpleTcpClient.htm>
- <http://www.java2s.com/Code/CSharp/Network/SimpleTcpServer.htm>

Packages used

- `using System;`
- `using System.Net;`
- `using System.Net.Sockets;`
- `using System.Text;`
- `using System.Threading;`